

Official Report: Python Static Analysis Security Script

Author: Solomon Mwaura

Date: April 27, 2026

Subject: Code Review and Enhancement of a Python Static Analysis
Tool for Security Vulnerabilities

1. Overview

This report provides a comprehensive review of the Python script designed for static analysis to identify potential vulnerabilities in software controlling production equipment within a manufacturing facility. The script focuses on detecting the use of unsafe functions that could lead to command injection or insecure deserialization.

Script Objectives:

- Recursively collect Python source files from a specified directory.
- Parse Python code into Abstract Syntax Trees (ASTs).

- Identify calls to a predefined list of unsafe functions.
- Report potential vulnerabilities found, including file path and line number.

2. Code Quality Assessment

2.1. Readability and Structure

- The script is well-organized into logical functions (`collect_source_files`, `parse_source_code`, `is_vulnerable_function`, `analyze_ast`, `main`).
- Function names are descriptive, and comments explain the purpose of each section.
- The use of AST parsing (`ast` module) is an appropriate and robust method for static code analysis, avoiding brittle regex-based approaches.

2.2. Error Handling

- The `parse_source_code` function includes a `try-except` block to catch `Exception` during AST parsing, preventing crashes due to malformed Python files.
- **Improvement:** The `except Exception as e: in parse_source_code` is broad. While it handles all parsing errors, it might be more informative to catch specific `SyntaxError` or `ValueError` if applicable, though `Exception` is often acceptable for AST parsing due to the variety of potential issues.
- The `main` function checks `if not os.path.isdir(target_directory) :` to validate the input directory, preventing errors from invalid paths.

2.3. Efficiency

- **collect_source_files:** `os.walk` is an efficient way to recursively find files.
- **parse_source_code:** Reading the entire file into memory (`source.read()`) before parsing is generally fine for typical source code file sizes. For extremely large single files (e.g., auto-generated code), it could be a concern, but this is less common for Python source.
- **analyze_ast:** `ast.walk(tree)` efficiently traverses the AST. The `is_vulnerable_function` check is performed for each node, which is standard for AST-based analysis.

2.4. Security and Vulnerability Detection Logic

- The approach of identifying unsafe functions via AST traversal is robust against simple obfuscation (e.g., changing whitespace, comments) that regex-based scanning would miss.
- The `UNSAFE_FUNCTIONS` list includes common culprits for command injection (`os.system`, `subprocess.call`) and insecure deserialization (`pickle.loads`), as well as code execution (`eval`, `exec`). The inclusion of `input` is also good as it can be a source of untrusted data.
- **Improvement (Completeness of `UNSAFE_FUNCTIONS`):**
The list could be expanded to include more functions or modules commonly associated with security risks, such as:
 - `subprocess.run`, `subprocess.Popen` (other ways to execute commands)
 - `yaml.load` (if `yaml` is used, `yaml.safe_load` should be preferred)
 - `json.loads` (if used with untrusted data and `object_hook`)
 - `marshal.load`, `shelve` (other deserialization risks)

- `mako.template.Template`,
`jinja2.Environment` (template injection risks if not configured securely)
- Direct use of `__import__` or `getattr` with untrusted input.
- **Improvement (Contextual Analysis):** The script currently flags any use of these functions. In a real penetration test, some uses might be legitimate (e.g., `eval` on trusted, controlled input). A more advanced tool would perform data flow analysis to determine if untrusted input reaches these functions, reducing false positives. However, for a junior penetration tester, flagging all occurrences is a safer initial approach.
- **Improvement (Reporting Details):** The report currently shows `line` number. Adding the column number (`col_offset`) would provide more precise location information.

2.5. User Experience and Reporting

- The script provides clear console output for scanning progress and a summary report.
 - **Improvement (Output Format):** The report is printed directly to the console. For larger projects or integration into CI/CD pipelines, generating the report in a structured format (e.g., JSON, XML, or a dedicated text file) would be beneficial. This would allow for easier parsing and aggregation of results.
 - **Improvement (Command-Line Arguments):** Using `input()` for the target directory is interactive but less flexible for automation. `argparse` would allow specifying the directory directly when running the script.
-

3. Identified Issues and Proposed Solutions

Category	Issue	Impact	Proposed Solution
Robustness	Broad <code>except Exception in parse_source_code</code>	May obscure specific parsing errors	Consider catching <code>SyntaxError</code> or <code>ValueError</code> specifically, or keep <code>Exception</code> if broadness is intended.
Completeness	<code>UNSAFE_FUNCTIONS</code> list is not exhaustive	May miss other common vulnerability patterns	Expand the list to include more known risky functions/modules (e.g., <code>subprocess.run</code> , <code>yaml.load</code>).
Precision	Flags any use of unsafe functions (high false positive potential)	May report legitimate uses as vulnerabilities	For advanced analysis, implement data flow analysis to track untrusted input. For this level, flagging all is acceptable.
Reporting	Console-only report output	Difficult to save, share, or integrate into	Implement <code>argparse</code> for output format (e.g.,

Category	Issue	Impact	Proposed Solution
		automated workflows	text, JSON) and write to file.
Usability	Interactive input for target directory	Less flexible for automation/scripting	Use <code>argparse</code> for command-line specification of the target directory.
Reporting Detail	Lacks column number in issue report	Less precise location of vulnerability	Include <code>col_offset</code> from AST node in the issue report.

4. Improved Version of the Code

Below is a revised version of the script incorporating the suggested improvements, particularly focusing on robustness, completeness of vulnerability patterns, and flexibility with command-line arguments and output formats.

```
#!/usr/bin/env python3
import os
import ast
import argparse
import json
import sys

# Scenario: Junior Penetration Tester - Manufacturing Facility
# Mission: Identify vulnerabilities in software controlling production
```

```

# 1. Function to recursively search for Python files
def collect_source_files(directory):
    """
    Recursively searches a directory for Python files (.py).
    """
    python_files = []
    for root, dirs, files in os.walk(directory):
        for file in files:
            if file.endswith(".py"):
                python_files.append(os.path.join(root, file))
    return python_files

# 2. Function to parse code into an Abstract Syntax Tree (AST)
def parse_source_code(file_path):
    """
    Reads a source file and converts it into an AST.
    Handles I/O and parsing errors.
    """
    try:
        with open(file_path, "r", encoding="utf-8") as source:
            return ast.parse(source.read(), filename=file_path)
    except FileNotFoundError:
        print(f"Error: File not found at {file_path}", file=sys.stderr)
        return None
    except SyntaxError as e:
        print(f"Error parsing syntax in {file_path}: {e}", file=sys.stderr)
        return None
    except Exception as e:
        print(f"An unexpected error occurred parsing {file_path}: {e}", file=sys.stderr)
        return None

# 3. Define Vulnerability Patterns and Helper Function
# We focus on functions often used for Command Injection or Insecure Deserialization
UNSAFE_FUNCTIONS = {
    # Command Execution
    'eval', 'exec', 'os.system', 'subprocess.call', 'subprocess.run',
    # Insecure Deserialization
    'pickle.loads', 'pickle.load', 'yaml.load', 'marshal.load', 'shelve.open',
    # Input Sources (often lead to vulnerabilities if not sanitized)
    'input',
    # Other potentially risky functions/modules

```

```

'os.popen', 'os.execv', 'os.execl', 'os.execl', 'os.execvp', 'os.e
'shutil.rmtree', # Can be dangerous if path is untrusted
'tempfile.mkstemp', 'tempfile.mkdtemp', # Can be dangerous if perm
'ctypes.CDLL', 'ctypes.WinDLL', 'ctypes.PyDLL', # Loading arbitrary
'xml.etree.ElementTree.parse', 'xml.sax.parse', # XML External Enti
'json.loads', # If used with object_hook and untrusted data
}

def is_vulnerable_function(node):
    """
    Checks AST nodes for the use of unsafe functions.
    """
    if isinstance(node, ast.Call):
        # Handle simple calls like eval()
        if isinstance(node.func, ast.Name):
            if node.func.id in UNSAFE_FUNCTIONS:
                return True, node.func.id

        # Handle attribute calls like os.system() or pickle.loads()
        elif isinstance(node.func, ast.Attribute):
            # Extract the full name, e.g., 'os.system', 'pickle.loads'
            if isinstance(node.func.value, ast.Name):
                full_func_name = f"{node.func.value.id}.{node.func.attri
                if full_func_name in UNSAFE_FUNCTIONS:
                    return True, full_func_name

            # Handle nested attributes like 'module.submodule.func'
            elif isinstance(node.func.value, ast.Attribute):
                # This part can be expanded for deeper nesting if needed
                # For simplicity, we'll just check the immediate parent
                # A more robust solution would recursively build the fu
                pass # Current UNSAFE_FUNCTIONS are mostly module.func

    return False, None

# 4. Function to traverse and analyze the AST
def analyze_ast(tree, file_path):
    """
    Analyzes the AST of a source file for potential vulnerabilities.
    Returns a list of found issues.
    """
    issues = []
    for node in ast.walk(tree):

```



```

        vulnerable, func_name = is_vulnerable_function(node)
    if vulnerable:
        issue = {
            "file": file_path,
            "line": getattr(node, 'lineno', 'unknown'),
            "col": getattr(node, 'col_offset', 'unknown'), # Added
            "issue": f"Use of unsafe function '{func_name}'"
        }
        issues.append(issue)
    return issues

# 5. Main function to tie everything together
def main():
    parser = argparse.ArgumentParser(description="Perform static analysis")
    parser.add_argument("target_directory", help="The directory path to scan")
    parser.add_argument("-o", "--output", choices=['text', 'json'], default='text',
                        help="Output format for the report (default: text)")
    parser.add_argument("-f", "--output_file", help="Optional: Path to output file")
    args = parser.parse_args()

    if not os.path.isdir(args.target_directory):
        print(f"Error: Invalid directory path: {args.target_directory}")
        sys.exit(1)

    print(f"\nScanning directory: {args.target_directory}")
    source_files = collect_source_files(args.target_directory)

    if not source_files:
        print("No Python files found in the target directory.")
        sys.exit(0)

    all_issues = []
    for file_path in source_files:
        print(f"  Analyzing {file_path}...")
        tree = parse_source_code(file_path)
        if tree:
            file_issues = analyze_ast(tree, file_path)
            all_issues.extend(file_issues)

    # Reporting the findings
    report_content = ""
    if args.output == 'json':

```

```

        report_content = json.dumps(all_issues, indent=4)
    else: # text format
        report_content += "\n" + "="*50 + "\n"
        report_content += "STATIC ANALYSIS SECURITY REPORT\n"
        report_content += "="*50 + "\n"

    if not all_issues:
        report_content += "No vulnerabilities detected.\n"
    else:
        report_content += f"Found {len(all_issues)} potential vulne
        for issue in all_issues:
            report_content += f"FILE: {issue['file']}\n"
            report_content += f"LINE: {issue['line']}, COL: {issue
            report_content += f"ISSUE: {issue['issue']}\n"
            report_content += "-" * 30 + "\n"

    if args.output_file:
        try:
            with open(args.output_file, 'w', encoding='utf-8') as f:
                f.write(report_content)
            print(f"Report saved to {args.output_file}")
        except IOError as e:
            print(f"Error saving report to {args.output_file}: {e}", f
    else:
        print(report_content)

if __name__ == "__main__":
    main()

```

5. Conclusion

The provided script offers a solid foundation for static analysis of Python code to identify security vulnerabilities. By leveraging AST parsing, it provides a more robust approach than simple text-based scanning. The enhancements introduced, including a more comprehensive list of unsafe functions, improved error handling,

and flexible reporting options, significantly increase its utility for penetration testers and security analysts. While static analysis has inherent limitations (e.g., false positives, inability to detect runtime vulnerabilities), this improved tool serves as an excellent first line of defense in identifying common coding pitfalls that could lead to serious security flaws in production systems.